

Resolución del problema del
productor-consumidor con mutexes

Nicolás Corbal Capas

Hugo Veiga Couselo

Introducción

La concurrencia es una característica fundamental en los sistemas operativos modernos y un factor clave para mejorar el rendimiento. Sin embargo, el acceso concurrente a recursos compartidos hace necesario el uso de mecanismos de sincronización. Entre estos mecanismos destacan los mutexes y las variables de condición, que permiten coordinar la ejecución de múltiples hilos, garantizando la exclusión mutua en regiones críticas y evitando la espera activa, así no penalizando el rendimiento con cambios de contexto innecesarios o comprobaciones redundantes.

Uno de los problemas clásicos en este ámbito es el del productor-consumidor. En este escenario, un hilo productor genera datos que se almacenan en un buffer compartido, mientras que un hilo consumidor los extrae y procesa. Existen diversas soluciones a este problema, como el uso de semáforos o monitores. En este trabajo se analiza el uso de mutexes y variables de condición, valorando sus ventajas y desventajas para un futuro análisis comparativo con otras soluciones.

En el presente informe se documenta la implementación de una solución al problema productor-consumidor utilizando mutexes y variables de condición en lenguaje C, adaptando el código visto en clase para manejar un buffer de tamaño arbitrario y simulando diferentes velocidades de ejecución mediante llamadas a la función `sleep()`. Se analiza el funcionamiento de la solución, las decisiones de diseño adoptadas y las conclusiones obtenidas tras su ejecución.

Forma de ejecución

Primeramente, para compilar el código un ejemplo de comando de compilación usando el compilador gcc es:

```
gcc -Wall -I ./ con_mutex.c buffer.c -o ejecutable
```

Siendo el ejecutable ejecutado simplemente con:

```
./ejecutable
```

Funcionamiento del código

Las responsabilidades el código se reparten entre 3 archivos. El archivo `buffer.h` de cabecera, `buffer.c` con código de producción-consumo y `con_mutex.c` con la lógica funcional de la solución.

El archivo de cabecera `buffer.h` define las constantes de tamaño de buffer, máximo de iteraciones y divide las iteraciones en 3 intervalos para introducir distintas velocidades. Además, tiene la estructura principal de buffer con información como las posiciones de la cola o el número de datos en el buffer.

El archivo `buffer.c` incluye las funciones de producción del item que lee de un archivo dado. La función `insert_item()` no solo inserta el item en el buffer, sino que también espera dentro de la región crítica, ese comportamiento se da para favorecer condiciones de carreras y ver que con la implementación dada no suceden, asegurando la robustez del código. La función `remove_item()` realiza la análoga función, con con la espera también dentro de la región crítica. Además, tiene como variables globales a los mutexes y variables de condición para acceso fácil de los hilos.

El archivo `con_mutex` contiene la lógica esencial. La función `main` inicializa las variables como el buffer y las variables de límites de la cola, así como la cuenta, posteriormente lanza los hilos consumidor y productor y espera a que terminen y luego libera la variable de condición y mutex.

```
item = produce_item(arq);
if(!item) break;
pthread_mutex_lock(&mutex);
while(args->cantidad>=N) pthread_cond_wait(&condp,&mutex);
struct timespec ts = {0, 1000000000}; // 100 ms
nanosleep(&ts, NULL);
insert_item(item, args);
pthread_cond_signal(&condc);
pthread_mutex_unlock(&mutex);
```

El hilo productor realiza iterativamente la siguiente secuencia produce un item (leyendo del archivo), ejecuta `pthread_mutex_lock()` función que comprueba el valor del mutex, si vale 0 se bloquea y si vale 1 baja su valor a 0 y continua, como estas operaciones son atómicas asegura

la exclusión mutua y que aquel que va a acceder al buffer sea el único en acceder. Comprueba si la cantidad de datos es igual o mayor a N, si no lo es, hay hueco para escribir, entonces inserta el elemento en el buffer y suma la cantidad al sumador. Si valiese N no puede escribir, pero tampoco otros dado que tiene el mutex. Así ejecuta `pthread_cond_wait()` que realiza atómicamente o siguiente, desbloquea el mutex para el consumidor pueda acceder a la región y bloquea el hilo, al despertar vuelve a solicitar el uso del mutex. Aquel que lo despierta es el consumidor después de consumir, así nos aseguramos de que no haya interbloqueos. Por último, si las iteraciones no se consumiesen enteramente, se añade un centinela que marca el final para el consumidor y así no quede bloqueado.

```
pthread_mutex_lock(&mutex);
while(args->cantidad==0) pthread_cond_wait(&condc,&mutex);
struct timespec ts = {0, 1000000000}; // 100 ms
nanosleep(&ts, NULL);
item = remove_item(args);
pthread_cond_signal(&condp);
pthread_mutex_unlock(&mutex);
```

El hilo consumidor realiza lo análogo. Adquiere el mutex, luego comprueba si puede consumir y si no usa la variable de condición en pro de activar al productor, si no consume y aumenta la suma. Finalmente, o se terminan las iteraciones o lee el 0 provocando que deje de consumir y finalice. Cabe destacar que la variable de condición es distinta, esto dota al problema de escalabilidad pues dados varios productores y consumidores puede que, si queremos despertar a un productor, despertemos a un consumidor, teniendo que usar `pthread_cond_broadcast()` que es más ineficiente.

```
pc-hugo@hugo:~/Documentos/3_curso/2_cuatri/S0II/S0II_P3/ejercicio2$ ./con_mutex
Suma productor: 91
Suma productor: 91
pc-hugo@hugo:~/Documentos/3_curso/2_cuatri/S0II/S0II_P3/ejercicio2$ ./con_mutex
Suma productor: 55
Suma productor: 55
```

Los resultados del código fueron probados usando 2 archivos con números del 1 al 10 (suma 10) y otro son el sumatorio de $i=1..6$ de i^2 ; los resultados fueron los esperados. Como se muestra en la imagen.

Conclusiones

La solución implementada proporciona los resultados esperados. Se soluciona entonces el conflicto de la concurrencia que se tenía en la anterior implementación con espera activa. Garantizar la exclusión mutua, evitan la espera activa, los procesos que bloquean inmediatamente acceden a la región minimizando el impacto de la serialización y no se provoca inanición a ninguno de los 2 procesos.

Otras implementaciones como los monitores alejan del problema y en concreto al ser de alto nivel usan los mutexes como herramienta de bajo nivel provocando un sobre coste en la eficiencia. Los semáforos también tienen una eficiencia similar pero la solución requiere de más semáforos.

Esta comparativa evidencia un principio fundamental en sincronización: no existe un mecanismo universalmente óptimo, sino que la elección depende del problema concreto. Para exclusión mutua simple entre dos procesos, los mutexes son la solución natural y más eficiente. Los semáforos aportan mayor expresividad (conteo de recursos) pero resultan sobredimensionados para este caso, mientras que los monitores introducen abstracciones innecesarias que nos alejan del problema. Por ello, la solución basada en mutexes no solo es correcta, sino también la más adecuada desde el punto de vista del rendimiento y la claridad del código.